Travel Research Agency

A production-grade multi-agent system that plans trips using 6 specialist AI agents, orchestrated by LangGraph with guardrails, RAG, human-in-the-loop, and observability.

Built as a portfolio showcase demonstrating end-to-end AI engineering: from agent design through evaluation to Kubernetes deployment.

Architecture

```
                ┌──────────────┐
        ┌───────│ SUPERVISOR   │──────┐
        │       │ (Groq)       │      │
        │       └──────┬───────┘      │
        │              │              │
        ▼              ▼              ▼
┌─────────────┐  ┌─────────────┐  ┌─────────────┐
│ Researcher  │  │ Experience  │  │ Safety Analyst │
│ (Groq)      │  │ (Gemini)    │  │ (Gemini + RAG) │
└─────────────┘  └─────────────┘  └─────────────┘
                 │
                 ▼
              ┌──────────┐
              │ Budget   │
              │ (Groq)   │
              └──────┬───┘
                     │
                     ▼
              ┌──────────┐
              │ Writer   │
              │ (Claude) │
              └──────────┘
```

Key Technologies

Layer	Technology	Purpose
Orchestration	LangGraph	Multi-agent graph with state management
Models	Groq (Llama 3.3 70B), Gemini 2.0 Flash, Claude Sonnet 4.5	Multi-provider strategy
Tools	FastMCP	Model Context Protocol — 8 tools, 1 server
RAG	Qdrant + Jina v3	Travel advisory retrieval (asymmetric embedding)

Layer	Technology	Purpose
Guardrails	Rule-based + LLM-based	4-layer input/output validation
Memory	InMemorySaver + InMemoryStore	Short-term (thread) + long-term (user)
Human-in-the-Loop	LangGraph interrupt/resume	Budget check + final approval
API	FastAPI + SSE	Sync, streaming, and HITL resume endpoints
Evaluation	RAGAS + DeepEval	Faithfulness, completeness, tool correctness
Observability	LangSmith + Langfuse	Tracing, cost tracking, score attachment
Deployment	Docker + Kubernetes	Multi-stage build, HPA auto-scaling

Project Structure

```
travel-research-agency/
├── state.py              # TravelState TypedDict (shared state)
├── config.py             # Model instances + agent-tool mappings
├── routing.py            # Conditional edge functions
├── graph.py              # Full 14-node LangGraph assembly
├── run.py                # CLI demo with HITL resume
├── server.py             # FastAPI (5 endpoints + SSE streaming)
├── schemas.py            # Pydantic request/response models
├── requirements.txt
├── .env.example
├── Dockerfile            # Multi-stage production build
├── docker-compose.yml       # App + Postgres + Qdrant
│
├── agents/
│   ├── supervisor.py        # Routes to specialists (Groq)
│   ├── researcher.py        # Flights + hotels (Groq)
│   ├── experience.py        # Restaurants + activities (Gemini)
│   ├── safety_analyst.py    # Visa + advisories + RAG (Gemini)
│   ├── budget_optimizer.py  # Cost analysis + currency (Groq)
│   └── writer.py            # Final itinerary (Claude)
│
├── tools/
│   └── mcp_server.py        # FastMCP server with 8 tools
│
```

```
├── rag/
│   ├── documents.py        # Travel advisory data
│   ├── ingest.py           # Chunk → embed → Qdrant
│   └── retriever.py        # query_points + Jina v3
│
├── guardrails/
│   ├── input_guardrails.py   # Topic, length, PII (rule-based)
│   ├── injection_guard.py    # Prompt injection (LLM-based)
│   ├── factuality_guard.py   # RAG grounding check (LLM-based)
│   └── output_guardrails.py  # Itinerary completeness (rule-based)
│
├── memory/
│   └── user_context.py       # Load/save user preferences
│
├── hitl/
│   ├── budget_check.py       # Interrupt if over budget
│   └── final_approval.py     # Interrupt for itinerary review
│
├── tests/
│   ├── test_guardrails.py    # Unit tests (no LLM needed)
│   ├── test_rag_eval.py      # RAGAS + DeepEval hallucination
│   └── test_e2e.py           # GEval, tool correctness, bias, budget
│
└── k8s/
    ├── deployment.yaml       # 3 replicas, health probes, resource limits
    ├── service.yaml          # LoadBalancer
    ├── secret.yaml           # API keys
    └── hpa.yaml              # Auto-scale 3→10 pods at 70% CPU
```

Quick Start

1. Setup

```bash
git clone <repo-url> && cd travel-research-agency
python3.12 -m venv .venv && source .venv/bin/activate
uv pip install -r requirements.txt
cp .env.example .env  # fill in your API keys
```

2. Ingest RAG Documents

```bash
```

```
python -m rag.ingest
```

3. Start MCP Tool Server

```bash
python tools/mcp_server.py
# Serves 8 tools at http://localhost:8001/mcp
```

4. Run CLI Demo

```bash
python run.py
# Runs full pipeline with HITL interrupt/resume
```

5. Run FastAPI Server

```bash
uvicorn server:app --host 0.0.0.0 --port 8000 --reload
```

6. Docker Compose (Full Stack)

```bash
docker compose up -d
# Starts: FastAPI app + Postgres + Qdrant
curl http://localhost:8000/health
```

API Endpoints

Method	Path	Description
GET	`/health`	Readiness check
POST	`/plan`	Sync trip planning
POST	`/stream`	SSE real-time streaming
POST	`/resume`	Resume from HITL interrupt
GET	`/status/{thread_id}`	Poll trip status

Example: Plan a Trip

```bash
curl -X POST http://localhost:8000/plan \
  -H "Content-Type: application/json" \
  -d '{"query": "Plan a 7-day trip to Tokyo", "budget": 3000}'
```

Example: Resume from Interrupt

```bash
curl -X POST http://localhost:8000/resume \
  -H "Content-Type: application/json" \
  -d '{"thread_id": "abc-123", "decision": "approve"}'
```

Graph Flow

```
START → load_user_prefs → input_guard → injection_guard → SUPERVISOR
                                        |
                  ┌──────────────────────────────────────┤
                  ▼          ▼          ▼
              researcher  experience  safety_analyst
                  |          |          |
                  └── supervisor ──┘      factuality_guard
                                  | retry (max 2)
                                  ▼
                              supervisor
                                  |
                            budget_optimizer
                                  |
                            budget_check (HITL)
                                  |
                               writer
                                  |
                            final_approval (HITL)
                                  |
                             output_guard
                                  |
                            save_preferences → END
```

Three routing mechanisms coexist:

1. **Fixed edges** — deterministic: researcher → supervisor
2. **Conditional edges** — state-based: guardrail pass/block, HITL decisions
3. **Command routing** — dynamic: supervisor picks the next agent at runtime

Evaluation

Unit Tests (no LLM required)

```bash
pytest tests/test_guardrails.py -v
```

RAG Evaluation (requires OPENAI_API_KEY)

```bash
pytest tests/test_rag_eval.py -v
```

End-to-End Evaluation

```bash
deepeval test run tests/test_e2e.py
```

Metrics Tested

Metric	Framework	What It Measures
Faithfulness	RAGAS	Are claims grounded in retrieved context?
Response Relevancy	RAGAS	Is the answer about what was asked?
Context Precision	RAGAS	Are relevant chunks ranked at the top?
Itinerary Completeness	DeepEval GEval	Does output have all required sections?
Budget Accuracy	DeepEval GEval	Do costs sum correctly, stay in budget?
Tool Correctness	DeepEval	Did agents call the right tools?
Hallucination	DeepEval	Does output contradict the context?
Bias	DeepEval	Is the output fair across demographics?

Design Decisions

Why multi-provider? Each model is matched to its strength: Groq for speed (routing, search), Gemini for grounded reasoning (safety, RAG), Claude for prose quality (final itinerary).

Why MCP over direct tool binding? Decouples tools from agents. N agents × 1 protocol instead of N × M integrations. Tools can be tested independently, versioned separately, and swapped without touching agent code.

Why asymmetric embedding? Jina v3 uses `task="retrieval.passage"` for documents and `task="retrieval.query"` for queries. This outperforms symmetric embedding because queries and documents have different linguistic patterns.

Why 4-layer guardrails? Rule-based guards run first (fast, free) to catch obvious issues. LLM-based guards handle nuanced threats. The factuality guard creates a retry loop — if the Safety Analyst hallucinates, it re-runs with stricter grounding (max 2 retries).

Why dual memory? Checkpointer (short-term) enables HITL pause/resume within a conversation. Store (long-term) persists user preferences across conversations — seat preference, dietary needs, past trips.

License

MIT